제목 [장애물 통과 비행이 가능한 지능형 드론]

I. 서론

- 작품 요약 소개

저희 팀은 2025년 제8회 전국 대학교 인공지능 드론 경진 대회에 참가하기 위해 혁신적인 드론 시스템을 개발했습니다. 본 시스템은 Tello EDU 드론을 기반으로 하였으며, 내장 카메라를 개조하여 하단에 부착함으로써 라인 트레이싱 및 고도 인식 기술을 구현하여 장애물을 안전하게 감지하고 통과하도록 코딩되었습니다. 또한, 최종 착륙 지점인 흰색 스티로폼에 정확하게 착륙할 수 있도록 제작되었습니다. 저희 드론은 자율성, 안정성, 인식 능력을 종합적으로 보여주는 지능형 드론입니다.

- 기존 드론 대비 독창성 및 차별성, 강점 소개

저희 팀의 드론은 Python을 이용한 프로그래밍을 통해 자율 비행 제어 기술과 비전 포지셔닝 시스템을 탑재하여, 기존 드론과 비교할 때 매우 안정적인 자율 비행이 가능합니다. 저희 드론은 특히 여러 가지 안전 기능을 갖추고 있어 독창적이고 차별화된 성능을 제공합니다.

첫 번째로 라인 추적 기반 주행 기능 탑재입니다. 하단에 개조된 카메라를 통해 비전 기반 주행 루트 탐지 기능을 활용하여 바닥의 라인을 인식하고 드론이 경로를 이탈하지 않고 움직일 수 있도록 하였습니다.

두 번째로 고도 변화 인식 및 대응 기능으로 다양한 높이의 장애물 통과 가능입니다. 하단 카메라로 고도를 지속적으로 계산하여 다양한 높이의 장애물을 안전하게 회피하도록 만들었습니다.

세 번째로 정확한 목표 지점 착륙 인식을 위한 비전 기반 스티로폼 탐지 기술입니다. 착륙 지점인 흰색 스티로폼을 정확하게 인식하여 착륙할 수 있습니다.

네 번째로 시작부터 착륙까지 자동화된 구동 프로세스입니다. 드론을 직접 조종하는 대신 스스로 움직일 수 있도록 코드를 개발하여 시작점에서부터 장애물 회피, 도착, 착륙까지 자동화된 구동 프로세스를 구현했습니다.

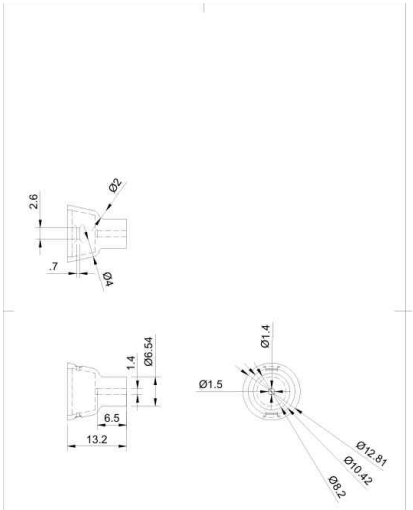
마지막으로 저가형 부품 기반 구현입니다. 저가형 부품 기반으로 구현 가능하여 확장성과 교육적 활용도가 높다는 강점을 가집니다.

이러한 파이썬 기반 소프트웨어적 기능들은 저희 드론이 기존의 드론과 차별화된 독창성과 안전성을 제공함을 입증합니다. 이로 인해 저희 팀의 작품은 코딩된 알고리즘을 통해 안정적이고 빠르게 장애물을 주파하고 착륙할 수 있습니다. 특히, 이번 프로젝트에서는 단순히 기능을 구현하는 것을 넘어, 센서 데이터 처리의 정확도를 높이고 비행 제어 로직을 최적화하기 위해 코드 레벨에서 매우 체계적이고 깊이 있는 접근 방식을 적용했습니다. 예를 들어, IMU(Inertial Measurement Unit) 센서의 초기 캘리브레이션을 통해 이륙 전 드론의 자세 안정성을 극대화하고, 비행 중 발생하는 미세한 오차를 지속적으로 보정하는 알고리즘을 구현하여 정밀한 제어를 가능하게 했습니다. 이는 드론의 불확실한 비행 환경에서도 높은 신뢰성을 보장하는 핵심 요소입니다.

II. 본론

- 구체적인 드론 강점 소개 (드론 및 제작에 사용된 부품 등의 사진 및 설계도면 첨부)

전방에 있던 드론의 카메라를 하단에 부착하여 Tello EDU 드론의 카메라를 활용해 바닥의 라인을 실시간으로 인식하고 판단하여 이동합니다. 또한, 장애물을 인식하여 고도를 조절하여 충돌을 방지하고 이동할 수 있습니다. 최종 착륙 시 안정성을 높이기 위해 드론 바닥에 3D 프린터로 제작한 받침대를 부착하고, 그 아래에 바늘을 달아 스티로폼 착륙 지점에 보다 정확하고 안정적으로 고정될 수 있도록 설계되었습니다. 이 받침대와 바늘은 3D 프린터를 이용하여 안정적으로 고정될 수 있도록 제작되었습니다.



(드론 바닥 받침대 3D프린터 설계도면)

2.1 드론 구성 내용 및 특징 소개

저희 팀 대회 드론은 기본 Tello EDU 드론을 이용하였습니다. 이 드론은 저렴한 가격에도 불구하고, 적절한 소프트웨어를 적용하면 매우 안정적인 비행 성능을 제공하여 제작 기체로 선정하게 되었습니다. Tello EDU 드론에는 가속도계, 자이로스코프, 기압계, 거리계(ToF 센서의 일종), 그리고 하단 비전 시스템(옵티컬 플로우 센서 포함)과 같은 센서가 내장되어 있어, 드론의 자세, 움직임, 위치 등을 정밀하게 감지할 수 있습니다.

본 드론 시스템은 소프트웨어와 하드웨어 양측을 모두 수정하여 비행이 가능하도록 구성되었습니다.

소프트웨어 측면: 제어 코드를 전면 수정하여 사람의 조작 없이도 드론이 자동으로 움직일 수 있도록 구현했습니다. 특히, moving_event, landing_event, moving_queue, landing_queue, line_queue와 같은 스레딩 이벤트와 큐를 적극적으로 활용하여 비디오 스트리밍과 드론 이동 로직을 비동기적으로 처리하고, 데이터 흐름을 체계적으로 관리했습니다. 이는 실시간 환경 변화에 대한 드론의 반응성과 안정성을 크게 향상시켰습니다.

하드웨어 측면: 드론에 내장된 카메라를 하단으로 개조하여 배치함으로써 높낮이와 검은색 선을 인식해 라인트레이싱을 이용할 수 있도록 했습니다.
이러한 기능을 이용하여 드론은 바닥의 검은 선을 라인트레이싱하여 이동하고, 장애물의 높낮이를 감지하여 스스로 고도를 조절하여 안전하게 통과할 수 있습니다.

2.2 드론 구동을 위한 센서 종류 및 활용법 소개
드론 구동을 위해 사용하는 다양한 센서들에 대해 설명드리겠습니다.

IMU (Inertial Measurement Unit) 센서: Tello EDU 드론에 내장된 핵심 센서로, 가속도계와 자이로스코프가 포함되어 있습니다.
가속도계 (Accelerometer)란 드론의 주행 중 가속도를 측정하는 필수적인 센서로, 비행 중 발생하는 가속과 감속을 정확히 감지합니다. 이를 통해 드론의 수직 이동을 정밀하게 제어하고, 비행 중 안정성을 유지하는 데 중요한 역할을 합니다.

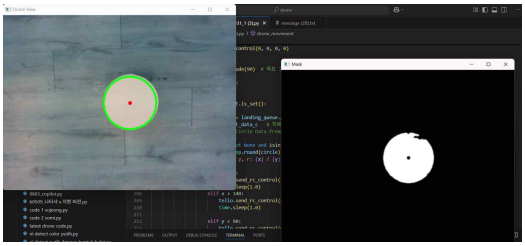
자이로스코프 (Gyroscope)란 드론의 회전 속도와 방향을 감지하는 역할을 하는 센서입니다. 드론의 자세와 회전을 정밀하게 조절하는 데 필요한 데이터를 제공합니다. . 저희는 calibrate_imu 함수를 통해 이륙 전 IMU 센서를 캘리브레이션하여 드론의 초기 자세를 안정화하고, 비행 중 발생할 수 있는 오차를 최소화했습니다. 이는 비행의 정밀도를 높이는 데 결정적인 기여를 합니다.

비전 카메라 (개조된 내장 카메라): 드론 하단에 개조하여 부착된 카메라를 이용하여 바닥의 검은 선과 흰색 원형 스티로폼 착륙 지점을 인식합니다. 이를 위해 OpenCV 컨투어 분석 기술을 사용하여 드론을 움직일 수 있게 하였습니다. 특히, detect_white_object 함수에서는 넓은 HSV 범위와 medianBlur, GaussianBlur를 적용하여 흰색 객체 인식을 강화했고, process_landing_position에서는 유효한 마스크 데이터가 없을 경우 직전 유효 데이터를 사용하여 비전 신호의 연속성을 확보했습니다. 또한, detect_circular_object 함수에서는 cv2.HoughCircles의 param1과 param2 값을 정밀하게 튜닝하고 minRadius, maxRadius를 설정하여 원형 객체 검출의 정확도를 극대화했습니다.

ToF (Time-of-Flight) 센서: 지면과 드론 간의 거리를 측정하는 센서입니다. 측정된 값을 이용하여 장애물 통과 시 드론의 고도를 조절하거나 착륙 시 높이를 판단하여 안정적인 비행을 할 수 있도록 했습니다.
maintain_altitude 함수에서는 ToF 센서 값을 기반으로 드론이 목표 고도를 벗어날 경우 정교하게 상승 또는 하강 제어를 수행하여 일정한 고도를 유지하도록 구현했습니다.

2.3 자율 주행 드론을 위한 프로그램 알고리즘 소개 (알고리즘 흐름도 포함)
자율 비행 드론을 구현하기 위한 프로그램 알고리즘은 드론의 초기화 및 안정화 단계에서 시작됩니다. 이 단계에서는 센서 초기화 및 캘리브레이션을 포함하여, 드론이 안정적으로 비행할 수 있도록 준비합니다.

OpenCV를 활용한 이미지 처리:
드론의 자율 비행을 위한 첫 번째 핵심 기술로는 OpenCV(Open Source Computer Vision Library)가 있습니다. OpenCV는 이미지 및 비디오 처리, 인식, 추적, 분류와 같은 다양한 컴퓨터 비전 문제를 해결하기 위한 도구들을 제공합니다. 저희 팀은 특히 OpenCV를 이용한 색 인식 기술에 주력하여 접근하였는데 , 이는 이미지나 비디오에서 특정 색상을 감지하고 추적하는 데 사용됩니다. OpenCV를 사용하면 색 공간 변환, 마스크 생성, 컨투어 검출 등을 통해 효과적으로 색 인식을 구현할 수 있습니다. 대부분의 이미지나 비디오는 RGB 색 공간에서 표현되지만, 색 인식을 위해서는 HSV(Hue, Saturation, Value) 색 공간으로 변환하는 것이 유리합니다. 아래는 드론이 실제 원을 인식한 사진입니다.
비전 포지셔닝 시스템(VPS):
두 번째 기술로는 비전 포지셔닝 시스템(VPS)입니다. 개조된 하단 카메라를 이용한 하향 비전 시스템과 ToF 센서를 사용하여 구현한 비전 포지셔닝 시스템은 드론 기체 하부에 장착된 카메라와 센서를 통해 지면을 스캔하고, 이를 통해 드론 기체의 위치를 실시간으로 계산하는 기술입니다. VPS는 주행 중 드론



이 일정한 고도를 유지하거나 설정한 주행 환경을 벗어나지 않게 위치를 고정할 수 있게 도와줍니다.

IMU(Inertial Measurement Unit):
세 번째 기술로는 IMU(Inertial Measurement Unit)가 있습니다. IMU는 드론의 가속도와 회전 속도를 측정하여 드론의 자세를 감지하고 판단할 수 있게 도와주는 기술입니다. 이 기술을 통해 드론 기체의 균형을 유지하거나 기울기나 회전을 판단하고 정확하게 제어할 수 있게 도와줍니다. IMU는 드론의 전반적인 비행 안정성에 중요한 역할을 합니다.

실제 IMU완료 사진

```
[INFO] tello.py - 462 - Response attitude?: 'pitch:0;roll:-1;yaw:-1;'
{'pitch': 0, 'roll': -1, 'yaw': -1}
[INFO] tello.py - 438 - Send command: 'attitude?'
[INFO] tello.py - 462 - Response attitude?: 'pitch:0;roll:-1;yaw:-1;'
{'pitch': 0, 'roll': -1, 'yaw': -1}
[INFO] tello.py - 438 - Send command: 'attitude?'
[INFO] tello.py - 462 - Response attitude?: 'pitch:0;roll:-1;yaw:-1;'
{'pitch': 0, 'roll': -1, 'yaw': -1}
✅ 캘리브레이션 완료!
```

GPS와 GLONASS:
네 번째 기술로는 GPS와 GLONASS 입니다. 드론은 GPS 및 GLONASS 시스템을 통해 실시간으로 자신의 위치를 파악하고, 설정된 경로를 따라 비행할 수 있게 됩니다. 이를 통해 정확한 경로를 유지하며, 목적지까지 안전하게 도착할 수 있습니다. GLONASS는 GPS와 유사한 기능을 제공하며, 두 시스템을 함께 사용하면 드론의 위치 정확도와 신뢰성을 향상시킬 수 있습니다.

비행 제어 알고리즘: 비행 제어 알고리즘은 사용자 입력 또는 프로그램에 의한 목표 지점을 설정하고, 비행 경로를 계획 및 갱신하며, 드론의 속도, 방향, 높이 등을 조절하여 목표 지점으로 이동합니다.

비행 모드 및 기능: 자동 이륙 및 착륙 기능 구현, 장애물 회피 기능 구현, 자동 조종 및 자율 주행 기능 추가로 구성됩니다.

영상 처리: 하단에 개조된 카메라를 사용하여 비행 환경을 실시간으로 촬영하고, 컴퓨터 비전(OpenCV) 및 객체 지향 알고리즘을 통해 물체 감지, 타겟 트래킹, 경로 계획 등을 수행합니다.

프로그래밍 환경: 프로그래밍 환경으로는 Visual Studio Code (VSC)를 IDE로 사용하며, Python 3.9.6 버전과 opencv-python, djitellopy, numpy, socket 등의 패키지를 설치하여 파이썬 가상 환경을 구축합니다.

전체 알고리즘 흐름:
IMU 캘리브레이션: 이륙 전에 드론의 IMU 센서를 캘리브레이션하여 좌우 또는 전후로 밀리는 현상 없

이 안정적으로 이륙할 수 있도록 하였습니다. calibrate_imu 함수는 여러 샘플을 통해 정확한 평균 오프셋을 계산하여 드론의 초기 안정성을 극대화합니다.
자동 이륙 및 영상 스트리밍 시작: 캘리브레이션 완료 후 드론이 자동으로 이륙하고, 하단에 개조된 카메라를 통해 영상을 스트리밍합니다. 실시간 영상은 바닥의 흰색 객체 탐지를 위한 마스크 화면과 검정 라인 인식 및 거리 확인용 화면으로 나뉩니다. stream_video 함수는 cv2.waitKey(5)를 사용하여 사용자 입력을 실시간으로 감지하고, 비상 착륙(ord('e'))과 같은 예외 상황에 즉각적으로 대응할 수 있도록 설계되었습니다. 또한, 비전 처리 결과를 별도의 큐(moving_queue, landing_queue, line_queue)에 저장함으로써 메인 드론 제어 로직과의 효율적인 데이터 교환을 가능하게 합니다.
초기 대기 및 상승: time.sleep을 이용하여 짧은 시간 대기하며 기체 안정화를 진행한 뒤, 목표 고도인 145cm까지 상승합니다.
라인 추적 및 장애물 통과: 목표 고도까지 상승 후 드론은 검은 선을 확인하여 정렬하고, 선을 따라 이동하며 다음 과정을 거칩니다.
첫 번째: 해당 고도로 라인을 확인하며 직진 후 첫 번째 장애물을 통과합니다. align_drone_to_line 함수는 line_queue에서 실시간으로 검은 선의 중심 x좌표를 받아 드론을 중앙에 정렬시킵니다. 오차 범위 tolerance를 설정하여 불필요한 미세 조정을 줄이고, tello.send_rc_control을 통해 정확한 횡 이동 제어를 수행합니다.
두 번째: 첫 번째 장애물 통과 후 다음 장애물 통과를 위해 드론을 90cm로 고도 하강시킵니다.
세 번째: 하강 후 전진하여 다음 장애물을 통과합니다. 그 후 다음 장애물 통과를 위해 다시 145cm까지 드론을 상승시킨 후 전진합니다.
네 번째: 모든 장애물을 통과한 후 원형 인식 (HoughCircles)과 화이트 마스크 필터링을 통해 착륙 지점인 흰색 스티로폼을 감지합니다.
착륙: 드론은 착륙 마커의 위치와 면적을 계산한 뒤, 조건에 따라 좌우 및 전후로 위치를 조정하여 착륙 지점의 중앙에 정렬합니다. 드론이 최종적으로 조건을 만족하면 고도를 내려 부드럽고 안전하게 자동 착륙을 수행합니다. 착륙 과정에서는 moving_event와 landing_event를 사용하여 단계별로 제어를 전환하며, landing_queue에서 원형 객체(x, y, r 좌표)를 받아 드론의 위치를 정교하게 조정합니다. 특히, y 값(전후 위치)과 x 값(좌우 위치)을 기반으로 정밀 제어를 수행하고, 특정 범위 내에 들어왔을 때 landing_event를 트리거하여 최종 착륙을 진행하는 체계적인 로직을 구현했습니다.

라인 추적 및 장애물 통과: 목표 고도까지 상승 후 드론은 검은 선을 확인하여 정렬하고, 선을 따라 이동하며 다음 과정을 거칩니다

첫 번째: 해당 고도로 라인을 확인하며 직진 후 첫 번째 장애물을 통과합니다.
두 번째: 첫 번째 장애물 통과 후 다음 장애물 통과를 위해 드론을 90cm로 고도 하강시킵니다.
세 번째: 하강 후 전진하여 다음 장애물을 통과합니다. 그 후 다음 장애물 통과를 위해 다시 145cm까지 드론을 상승시킨 후 전진합니다.
네 번째: 모든 장애물을 통과한 후 원형 인식 (HoughCircles)과 화이트 마스크 필터링을 통해 착륙 지점인 흰색 스티로폼을 감지합니다.
착륙: 드론은 착륙 마커의 위치와 면적을 계산한 뒤, 조건에 따라 좌우 및 전후로 위치를 조정하여 착륙 지점의 중앙에 정렬합니다. 드론이 최종적으로 조건을 만족하면 고도를 내려 부드럽고 안전하게 자동 착륙을 수행합니다.

Ⅲ. 구현

- 대회 참가를 위한 기본 제작과정 소개

저희 팀은 대회 참가를 위해 다음과 같은 기본 제작 과정을 따랐습니다. 알고리즘을 어느 방식을 사용

하였는가와 어떻게 구현을 하였는가에 집중하여 기술하겠습니다.

드론 제어: djitellopy 라이브러리를 사용하여 드론의 전반적인 움직임을 제어 하였습니다.
카메라 개조 및 방향 조정: 드론의 내장 카메라를 개조하여 하단에 부착하고, 감지 정확도를 높이기 위해 카메라 방향을 조정했습니다.

착륙 핀 부착: 드론이 착륙 시 스티로폼 위에서 미끄러지지 않도록, 하단에 작은 핀을 부착했습니다.

착륙 코드 개발: 드론이 착륙 지점을 정확하게 감지하도록 착륙 코드를 우선적으로 개발했습니다.
장애물 통과 알고리즘 개발: 착륙 지점 감지 코드가 완성된 이후에는 장애물을 통과하는 드론의 이동 알고리즘을 개발했습니다.

미션 수행 알고리즘: 이번 대회의 임무를 수행하기 위한 기술은 크게 세 가지로 요약할 수 있습니다. 드론을 정밀하게 제어할 수 있는 기술, Computer Vision을 사용한 장애물 인식, 마찬가지로 Computer Vision을 사용한 착륙 지점 인식입니다. 이를 구현하기 위해서 저희는 OpenCV를 사용하는 방식을 채택하였습니다.
장애물 인식 기술: 카메라에 촬영된 장애물을 주변 환경과 분리해내기 위해 외곽선 검출 알고리즘(Canny 함수)을 사용하였습니다. findContours 함수를 이용하여 사각형 형태의 오브젝트들의 크기와 위치를 받아, 장애물이 여러 형태로 동시에 인식되더라도 회피하고자 하는 장애물을 판단하여 올바르게 주행할 수 있게 구현 되었습니다. 또한, 드론이 전진하면서 고도가 떨어지는 경향을 보정하기 위해 move_forward 함수 내에서 get_speed_correction 함수를 통해 배터리 잔량에 따른 속도 보정을 적용하는 등, 실제 비행 환경의 변수를 코드에 반영했습니다.
착륙지점 인식 기술: 허프 변환(Hough Transformation) 중에서도 "허프 원 변환"을 이용하였습니다. 인식되는 원의 크기를 고도별로 지정하는 방식을 선택하여 카메라에서 발생하는 노이즈로 인한 부정확한 인식을 해결했습니다. 착륙 위치로 이동하는 공식 또한 구현하였으며, 2024년 보고서에서 전면부에 위치한 카메라로 인한 오차를 해결하기 위해 카메라 위치 상수를 도입한 부분과는 달리, 저희는 하단 카메라의 특성에 맞춰 착륙 지점 유도 공식을 적용했습니다. 특히 detect_circular_object 함수 내에서 param1=50, param2=40, minRadius=30, maxRadius=80와 같이 cv2.HoughCircles의 매개변수를 세밀하게 조정하여 노이즈 환경에서도 안정적인 원형 검출이 가능하도록 최적화했습니다.

- 드론 제작 중 애로기술 및 해결과정 소개

드론 제작 과정에서 다음과 같은 애로사항과 해결 과정을 겪었습니다.

이륙 안정성 문제: 드론 하단에 핀과 개조된 카메라가 부착되면서 평평하지 않은 표면 때문에 드론이 부드럽게 이륙하지 못하는 문제가 발생했습니다.

해결 과정: 핀의 길이를 동일하게 맞추고, 드론이 안정적으로 이륙할 수 있도록 캘리브레이션 단계를 추가하여 문제를 해결했습니다. calibrate_imu 함수는 이륙 전 필수적으로 수행되어 드론의 초기 안정성을 확보하며, 이는 예측 불가능한 초기 이륙 오류를 줄이는 데 크게 기여했습니다.

인식 정확도 문제: 드론이 검정 선, 흰색 스티로폼, 장애물을 정확하게 인식하는 데 어려움이 있었습니다.

해결 과정: 개조된 카메라의 각도를 조정하여 드론이 하단을 인식하는 정확도를 향상시켰습니다. 또한, 비전 처리 파이프라인에서 medianBlur, GaussianBlur, adaptiveThreshold와 같은 이미지 전처리 기법을

다단계로 적용하고, HSV 색상 공간을 활용하여 환경 변화에 강인한 객체 인식을 구현했습니다. 큐에 최신 데이터를 유지하고 이전 유효 데이터를 활용하는 로직(latest_valid_mask_data)을 통해 인식률 저하로 인한 비행 불안정성을 최소화했습니다.

- 드론 및 제작에 사용된 부품 등, 사진 및 설계도면 첨부 (3D 프린터 활용 시 도면 소개)
2P에 첨부 하였습니다.

- 대회 참가 전 연습 주행 시 드론의 작업 능력 및 주행시간 등

총 5회 시도 중 4회에서 드론이 성공적으로 작동했습니다. 대회에서 제공된 배점표 기준에 나와 있는 모든 자율 주행 미션을 대회장과 동일한 환경으로 구성하고 시험 주행을 실시했습니다. 30회 정도 실시하였고 평균적인 결과는 이륙 전 드론에 전원을 인가했을 때의 시간부터 착륙 지점을 인식하여 착륙하는 것까지 약 50초 가량 소요 되었습니다.

배터리 잔량에 따른 안정성: 배터리 잔량이 100%에서 70% 사이일 때는 드론이 매우 안정적으로 움직였습니다.
배터리 저하 시 문제: 하지만 배터리가 60% 이하로 떨어지면, 드론이 착륙 지점까지 이동하지 못하거나 스티로폼 위에 정확히 착륙하지 못하는 현상이 발생했습니다. 이는 배터리 잔량에 따른 드론의 성능 저하를 시사하며, 향후 배터리 관리 시스템 개선의 필요성을 보여줍니다. 이러한 문제에 대응하기 위해 get_speed_correction 함수를 구현하여 배터리 잔량에 따른 드론의 전진 속도를 보정하도록 시도했습니다. 이는 비행 중 배터리 저하로 인한 성능 변화를 최소화하려는 저희 팀의 체계적인 접근 방식의 일환입니다.

Ⅳ. 결론 및 향후 연구 방향

- 참가 팀만의 고유한 자랑거리 소개

저희 팀은 기존 Tello EDU 드론의 하드웨어와 내장 센서를 주력으로 활용하면서도, 내장 카메라를 하단에 개조하여 배치함으로써 대회 경기장과 규정에 맞는 알고리즘을 구현했다는 점에서 고유한 자랑거리가 있습니다. 복잡한 하드웨어 개조 없이도 드론이 정밀하게 추적하고, 코딩된 알고리즘에 따라 장애물을 회피하거나 우회하여 목적지에 도달할 수 있는 능력을 갖추게 했습니다. 특히, Tello EDU 드론의 가속도계와 자이로스코프 같은 내부 센서를 적극 활용하여 비행 중 드론의 자세와 움직임을 정확히 파악했으며, 이를 기반으로 개발한 알고리즘은 드론이 실시간으로 장애물을 감지하고 경로를 수정하며 효율적으로 목적지에 도달할 수 있도록 했습니다. 저희 팀은 특히 비전 처리와 드론 제어 로직을 통합하는 과정에서 멀티스레딩과 큐를 활용하여 데이터 처리의 효율성과 실시간성을 극대화했습니다. video_thread와 movement_thread를 분리하여 비전 인식과 드론 조종을 독립적으로 수행하면서도, 공유 큐를 통해 데이터를 안전하게 교환하는 견고한 시스템을 구축했습니다. 이는 단순히 코드를 작성하는 것을 넘어, 시스템 아키텍처를 깊이 있게 고민하고 설계했다는 증거입니다. 내장 카메라의 개조와 소프트웨어적 기능 구현을 통해 드론의 기능을 상당히 확장시킬 수 있었으며, 이는 비용 효율적이고 실용적인 솔루션을 제시한 것입니다. 이러한 접근 방식을 통해 저희는 드론의 성능을 극대화하면서도, 대회 규정에 맞춘 정밀한 비행과 장애물 회피 능력을 성공적으로 구현할 수 있었습니다.

- 대회를 통해 얻게 된 지식 및 향후 참가팀원들의 계획

대회를 준비하면서 저희는 Python을 사용하여 드론의 비행 경로를 제어하고, 센서 데이터를 분석하여 인공지능 비행을 구현하는 과정에서 다양한 기술적 문제를 해결해야 했습니다. 특히, OpenCV 기술과 허프 확률 변환에 대해 심도 있게 공부했습니다. 이러한 기술들은 드론의 개조된 하단 카메라와 센서를

활용한 실시간 데이터 분석 및 처리에 중요한 역할을 했습니다. OpenCV를 사용하여 이미지 처리 및 분석을 수행함으로써 드론이 환경을 인식하고 반응할 수 있는 능력을 갖추게 되었고, 허프 확률 변환을 통해서는 영상에서 선을 검출하여 비행 경로를 정확히 추적하고 장애물을 회피하는 알고리즘을 구현할 수 있었습니다. 이 과정에서 모든 팀원들은 코딩 기술을 크게 향상시킬 수 있었으며, 실시간 데이터 분석 능력도 개발할 수 있었습니다.

특히, 실제 환경에서의 여러 테스트를 통해 이론과 실제 간의 차이를 체감할 수 있었습니다. 이론적으로 완벽해 보이는 알고리즘도 실제 환경에서는 다양한 변수와 예상치 못한 상황들로 인해 예상대로 작동하지 않는 경우가 많았습니다. 이러한 문제들을 여러 차례 경험하고 해결해 나가면서, 저희 팀의 문제 해결 능력도 크게 강화되었습니다.
코드 내에서 다양한 임계값(예: lower_white, upper_white, minDist, param1, param2, minRadius, maxRadius, tolerance)을 수없이 조정하고 테스트하여 최적의 성능을 이끌어낸 과정은 저희 팀이 이론적 지식을 실제 환경에 적용하고 최적화하는 데 얼마나 많은 노력을 기울였는지를 보여줍니다. 또한, 배터리 잔량에 따른 속도 보정과 같은 현실적인 문제 해결 시도는 저희가 단순한 코딩을 넘어 실제 드론 운용 환경을 깊이 이해하고 대응했음을 나타냅니다.

이번 대회 준비 과정은 저희 팀에게 단순한 경쟁을 넘어, 참여한 모든 인원이 끊임없이 배우고 성장할 수 있는 소중한 기회였습니다. 대회를 준비하며 겪은 도전과 실패, 그리고 성공은 저희 팀의 역량을 한층 더 발전시키는 데 큰 도움이 되었습니다. 앞으로도 저희는 이 경험을 바탕으로 더욱 도전하고 발전해 나가는 데 집중할 것입니다. 특히, OpenCV와 허프 확률 변환을 학습한 경험은 향후 저희의 인생과 경력에도 큰 영향을 미칠 것이며, 이러한 기술들을 익힘으로써 앞으로 다양한 분야에서 이미지 처리 및 분석 기술을 활용할 수 있는 능력을 갖추게 되었습니다. 이러한 경험을 발판 삼아, 끊임없이 기술을 향상시키고 더 나은 성과를 이루기 위해 노력할 것입니다.

참고문헌

- 대회준비를 위해 참고한 교재 혹은 사이트 소개
[1] 드론 AI 알고리즘, 센서, 프로그램 관련 교재
[2] YouTube 등 기타 웹 사이트를 통해 참고한 사이트 소개
<드론 프로그래밍>#1 DJI TELLO 드론과 필요 환경 구축 / 파이썬 https://things-voyager.tistory.com/68

파이썬+OpenCV를 이용한 객체 인식 드론 만들기
https://velog.io/@jmhwang49/%ED%8C%8C%EC%9D%B4%EC%8D%ACOpenCV%EB%A5%BC-%EC%9D%B4%EC%9A%A9%ED%95%9C-%EA%B0%9D%EC%B2%B4-%EC%9D%B8%EC%8B%9D-%EB%93%9C%EB%A1%A0-%EB%A7%8C%EB%93%A4%EA%B8%B0

DJI Tello 파이썬 프로그래밍하기 https://stupidly-honest.tistory.com/24
Python OpenCV 강좌 : 제 28강 - 직선 검출 https://076923.github.io/posts/Python-opencv-28/
Easy Programming of Tello Drone | Python OpenCV Object Tracking
https://www.youtube.com/watch?v=vDOKUHNdmKs&si=vvuSFqS3aWLIT6kS